

**CSA05 – DATABASE MANAGEMENT SYSTEMS****ASSIGNMENT REPORT****Library Management Database System**

Student Name	K.Sandhya
RegistrationNumber	192424061
Course	Database Management Systems
Course Code	CSA05
Academic Year	2024–2028
Submission Type	Individual Assignment

Course Outcomes

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.\ CO3: Analyze relational databases using functional dependencies, normalization, and key constraints to identify redundancy and ensure data integrity.

Bloom's Taxonomy: L5 – Evaluate, L6 – Create\ SDG Mapping: SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions

A university wants to develop a **Student Academic Management Database System** to efficiently manage student information, course registration, faculty details, examination marks, and academic performance. The existing system stores information in separate files, resulting in **data redundancy, inconsistency, difficult data retrieval, and limited security**.

Students are required to analyze the given case and design a **relational database solution** using DBMS concepts. The solution should include identification of entities and relationships, relational schema design, appropriate primary and foreign keys, and normalization of the relations.

The database should support operations such as **inserting, updating, deleting, and retrieving records**. Students must formulate SQL queries to generate useful academic reports, such as students enrolled in a particular course, students scoring above a specified mark, course-wise performance, and faculty-wise course allocation. They should

also apply **relational algebra operations** and create suitable **views** for frequently required reports.

The design should consider requirements such as **data integrity, consistency, efficient query processing, security, and scalability**.

Students should justify their design decisions and demonstrate the working database using an appropriate DBMS tool such as **MySQL, PostgreSQL, or Oracle**.



SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Detailed Assessment Rubrics

S.No.	Assessment Criterion	Performance Indicators	BT Level	Marks
1	Requirement Analysis and System Understanding	Identifies functional and non-functional requirements; analyzes existing file based problems such as redundancy, inconsistency, difficult retrieval, and security limitations; defines database scope and user requirements.	BT5	
2	Entity and Relationship Identification	Identifies appropriate entities such as Student, Course, Faculty, Examination, Marks, and Registration; determines attributes, cardinalities, participation constraints, and meaningful relationships; develops a suitable ER diagram.	BT5	
3	Relational Schema and Key Design	Converts the ER model into relational schemas; identifies suitable primary keys, foreign keys, candidate keys, and composite keys; establishes appropriate relationships and referential integrity constraints.	BT5	
4	Normalization of Relations	Analyzes functional dependencies; identifies insertion, deletion, and update anomalies; applies 1NF, 2NF, 3NF and, where appropriate, BCNF; justifies	BT6	

		decomposition and lossless dependency preserving design.		
5	Database Creation and Constraint Implementation	Implements tables using an appropriate DBMS; applies PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints; validates data integrity and consistency.	BT6	
6	DML Operations and Transaction Handling	Demonstrates INSERT, UPDATE, DELETE, and SELECT operations; maintains consistency during modifications; demonstrates appropriate transaction handling using COMMIT, ROLLBACK, or SAVEPOINT where applicable.	BT6	
7	SQL Query	Develops complex SQL queries using	BT6	



Development and
Academic Reports

filtering, joins, aggregate functions,
grouping, subqueries, and ordering to
generate reports such as course-wise
enrollment, students scoring above
specified marks, course-wise
performance, and faculty-wise
allocation.

8	Relational Algebra and Query Formulation	Formulates appropriate relational algebra expressions using selection, projection, union, intersection, difference, Cartesian product, and join operations for academic information retrieval requirements.	BT6	
9	Views and Database Security	Creates suitable views for frequently accessed academic reports; applies appropriate access restrictions, privileges, GRANT/REVOKE mechanisms, and considers protection of sensitive student and examination information.	BT6	
10	Query Processing and Performance Optimization	Evaluates query efficiency; identifies suitable indexes; analyzes execution/performance considerations; justifies indexing and query optimization strategies for efficient retrieval.	BT5	
11	Data Integrity, Consistency and Scalability Assessment	Evaluates entity and referential integrity; considers concurrency, consistency, backup/recovery, increasing student records, and future scalability of the database solution.	BT5	
12	Innovation and Database Design Creativity	Demonstrates innovative database features such as automated academic reports, triggers, stored procedures/functions, role-based access, performance monitoring, or other useful extensions.	BT6	



13	Technical Report Quality	Presents well-organized documentation containing requirements, ER diagram, relational schema, normalization, SQL queries, relational algebra, screenshots/results, references, testing, and discussion of design decisions.	BT6	
14	Oral Presentation	Demonstrates the working database	BT6	



SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

and Demonstration using MySQL, PostgreSQL, Oracle, or another suitable DBMS; explains design decisions clearly; answers technical questions and demonstrates database operations effectively.

	TOTAL			100
--	--------------	--	--	------------

Table of Contents

1. Problem Statement.....	3 2
2. Objective.....	3 3 3.
Requirements and Environment Used	3
3.1 3.1 Functional requirements.....	3
3.2 3.2 Non-functional requirements.....	3
3.3 3.3 Environment.....	4
4. Design / Proposed Solution.....	4
4.1 4.1 Entities and relationships.....	4
4.2 4.2 Relational schema	6
4.3 4.3 Design decisions.....	6
5. Algorithm, Pseudocode and Flowchart	6
5.1 5.1 Academic-report algorithm.....	6
5.2 5.2 Pseudocode for above-threshold marks.....	6
5.3 5.3 Flowchart	6
6. Implementation / Source Code.....	7
6.1 6.1 Table definition.....	7
6.2 6.2 View and trigger.....	7
6.3 6.3 Relational algebra.....	7
7. Test Cases and Expected / Actual Results.....	7 8
8. Execution Screenshots / Output	8 9 9.
Analysis and Discussion.....	8 10 10.
Conclusion	9 11 11.
Individual Contribution of Group Members.....	9 12 12.
References.....	9 13 13.
One-Page Write-up.....	10



A college library needs a Library Management Database to efficiently manage members, books, publishers, book copies, and issue/return transactions. The existing manual or unstructured process can lead to difficulties in tracking available copies, overdue books, member borrowing history, and fines. The proposed database will store member details, book and publisher information, individual book copies, and issue transactions in a structured relational database. It will identify the relationships among entities, maintain data integrity using keys and constraints, and reduce redundancy through normalization. The system will also support SQL queries for identifying overdue books and member wise borrowing, provide a view for overdue issues, and use suitable indexes on ISBN, Member_ID, and Due_Date to improve query performance. Transaction handling will ensure that book issue and return operations are completed accurately and consistently.

2. Objective

The main objective of the College Library Management Database is to develop a structured and efficient database system for managing members, books, publishers, book copies, and issue/return transactions. The system aims to maintain accurate and consistent library records, establish proper relationships using primary and foreign keys, reduce data redundancy through normalization, and efficiently track book availability, due dates, returns, and fines. It also supports SQL queries for identifying overdue books and member-wise borrowing, provides a view for overdue issues, and improves query performance through suitable indexes on ISBN, Member_ID, and Due_Date. Transaction handling is incorporated to ensure that book issue and return operations are completed accurately and maintain database consistency.

3. Requirements and Environment Used

3.1 Functional requirements

The Student Academic Management Database System should provide essential functions for efficiently managing student academic records. The system must allow authorized users to add, view, update, and delete student information such as student ID, name, department, semester, and contact details. It should support the management of course details, student course registrations, attendance records, internal assessment marks, examination marks, grades, and semester results.

The system should provide search and filtering facilities to retrieve student records based on student ID, department, course, or semester. It should also calculate and display academic performance such as total marks, grades, and CGPA where required. In addition, the system should generate useful academic reports, maintain data consistency through appropriate database



constraints, and ensure that only authorized users can access or modify sensitive academic

information.. The College Library Management Database should provide functions to manage member details, book information, publisher records, and individual book copies. The system must allow librarians to register and maintain member records, add and update books and publisher details, and track the status of each book copy. It should support issuing books to members by recording the issue date and due date, processing returned books by updating the return date and calculating applicable fines, and identifying overdue books. The database should also provide member-wise borrowing information and maintain complete issue history. A dedicated overdue issues view should be available for quick retrieval of pending returns. The system must maintain data integrity using primary keys, foreign keys, and appropriate constraints, while indexes on ISBN, Member_ID, and Due_Date should improve the performance of frequently used searches and queries.

3.2 Non- requirements

Requirement	Design response
-------------	-----------------

Performance	Indexes on ISBN, Member_ID, and Due_Date provide fast data retrieval.
Integrity	Primary and foreign keys maintain accurate library records
Security	Only authorized users can access library data.
Scalability	Supports increasing members, books, and transactions.
Reliability	Transactions and backups prevent data loss.
Privacy and ethics	Member details are protected from unauthorized access.



Component	Selected environment
DBMS	PostgreSQL 16 or later; SQL is written to be broadly relational and portable.
Client	pgAdmin 4 or psql command-line client.
Operating system	Windows, Linux, or macOS laboratory environment.
Language	SQL and PL/pgSQL for the trigger function.
Hardware	A standard laboratory computer with at least 4 GB RAM and 10 GB available storage.

1 4. Design / Proposed Solution

The proposed College Library Management Database is designed as a relational database to manage members, books, publishers, book copies, and borrowing transactions efficiently. The system uses separate tables for each entity and establishes relationships using primary and foreign keys. A member can issue multiple book copies, while each book can have multiple copies. Publishers can publish multiple books. The database records issue dates, due dates, return dates, and fines to track borrowing activities and overdue books. Normalization is applied to reduce data redundancy and improve consistency. SQL queries, views, indexes, and transaction handling are used to provide fast, reliable, and accurate library operations.

1.1 4.1 Entities and relationships

The College Library Management Database consists of five main entities: Member, Book, Publisher, Copy, and Issue. The Member entity stores details of library members, while Book stores information about books. Publisher maintains publisher details, and Copy represents individual physical copies of books. The Issue entity records book borrowing and returning transactions.

Relationships: A Publisher can publish many Books (1:N). A Book can have many Copies (1:N). A Member can have many Issues (1:N). A Copy can appear in many **Issue** records over time (1:N). These relationships are implemented using foreign keys to maintain data consistency.



Figure 1: Entity–Relationship diagram for the Library Management Database System.



Relation	Attributes and keys
----------	---------------------

MEMBER	Member_ID (PK), Member_Name, Department_ID, Email, Phone
BOOK	Book_ID (PK), ISBN, Title, Author, Publisher_ID (FK), Price
PUBLISHER	Publisher_ID (PK), Publisher_Name, Contact_No
COPY	Copy_ID (PK), Book_ID (FK), Accession_No, Status
ISSUE	Issue_ID (PK), Copy_ID (FK), Member_ID (FK), Issue_Date, Due_Date, Return_Date, Fine

1.3 4.3 Design decisions

The database is designed using a normalized relational model to reduce redundancy and maintain data consistency. Separate tables are created for members, books, publishers, copies, and issue transactions. Primary keys uniquely identify each record, while foreign keys establish relationships between related tables. The Copy table is separated from the Book table to manage multiple physical copies of the same book. Issue and return details are stored in the Issue table, including due dates and fines. Indexes on ISBN, Member_ID, and Due_Date are recommended for faster searches. Transactions are used during book issue and return to ensure reliable and consistent database updates.

5. Algorithm, Pseudocode and Flowchart

1.4 5.1 Academic-report algorithm

- Start the library management system.
- Connect to the library database.
- Accept the Member_ID and verify whether the member exists.
- Accept the Book_ID/Copy_ID and check whether the book copy is available. · If the copy is available, create a new issue record with Issue_Date and Due_Date. · Update the copy status from Available to Issued.
- When the book is returned, record the Return_Date.

- Compare the Return_Date with the Due_Date to determine whether the book is overdue.
- If overdue, calculate and store the applicable Fine.



CSA05 – Database Management Systems · Update the copy status from Issued to Available.

- Commit the transaction to permanently save the changes.
- Display the issue, return, and fine details.
- Stop the process..

1.1 5.2 Pseudocode for above-threshold marks

BEGIN

CONNECT to Library Database

INPUT Member_ID

IF Member_ID exists in MEMBER table THEN

INPUT Copy_ID

CHECK Copy status

IF Status = "Available" THEN

SET Issue_Date = Current Date

SET Due_Date = Issue_Date + Loan Period

INSERT issue details into ISSUE table

UPDATE COPY status to "Issued"

COMMIT transaction

DISPLAY "Book issued successfully"

ELSE

DISPLAY "Book copy is not available"

END IF

ELSE

DISPLAY "Member not found"

END IF

FOR book return:

INPUT Copy_ID

SET Return_Date = Current Date



FIND Due_Date from ISSUE table

IF Return_Date > Due_Date THEN

CALCULATE Fine

ELSE

SET Fine = 0

END IF

UPDATE ISSUE with Return_Date and Fine

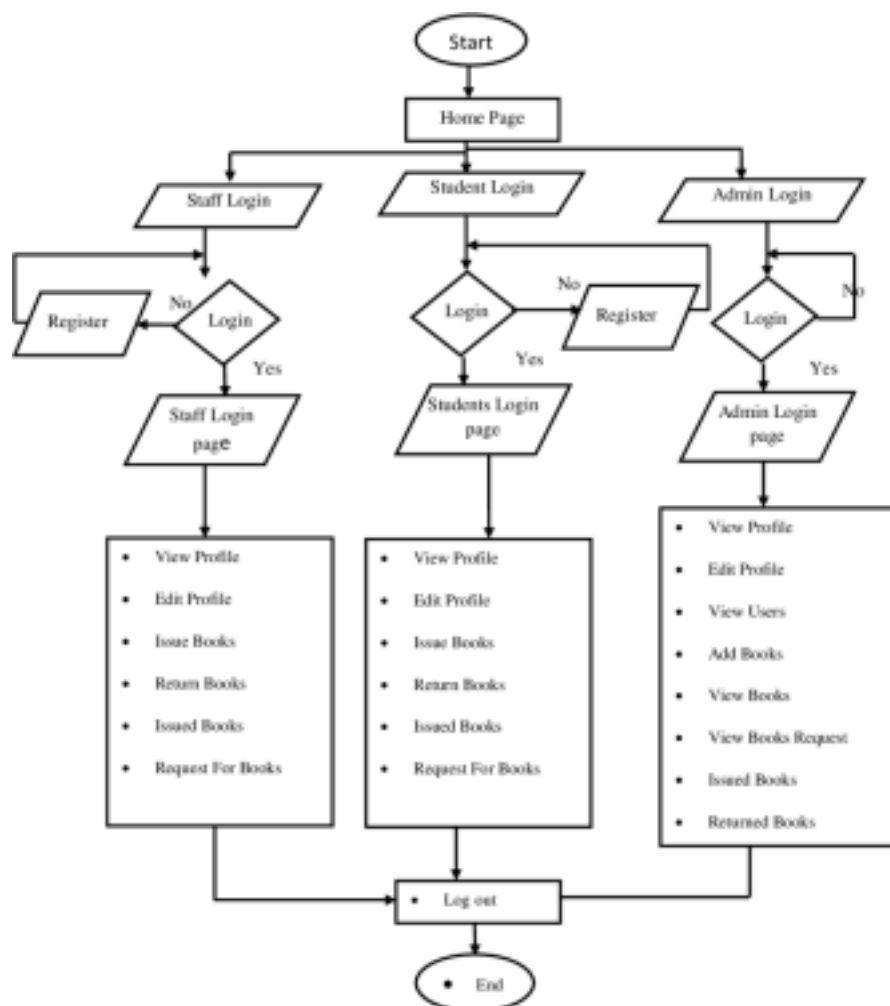
UPDATE COPY status to "Available"

COMMIT transaction

DISPLAY "Book returned successfully"

END

1.2 5.3 Flowchart





The College Library Management Database is implemented using SQL in PostgreSQL. The implementation includes table creation, primary and foreign key constraints, data insertion, book issue and return operations, overdue-book queries, member-wise borrowing queries, and an overdue issues view. Indexes are also created on ISBN, Member_ID, and Due_Date to improve query performance. Transaction commands such as BEGIN, COMMIT, and ROLLBACK are used to ensure reliable issue and return operations.

COLLEGE LIBRARY MANAGEMENT DATABASE

1. Create Tables

```
CREATE TABLE Publisher (
```

```
    Publisher_ID INT PRIMARY KEY,
```

```
    Publisher_Name VARCHAR(100) NOT NULL,
```

```
    Contact_No VARCHAR(15)
```

```
);
```

```
CREATE TABLE Member (
```

```
    Member_ID INT PRIMARY KEY,
```

```
    Member_Name VARCHAR(100) NOT NULL,
```

```
    Department_ID INT,
```

```
    Email VARCHAR(100) UNIQUE,
```

```
    Phone VARCHAR(15)
```

```
);
```

```
CREATE TABLE Book (
```

```
    Book_ID INT PRIMARY KEY,
```

```
    ISBN VARCHAR(20) UNIQUE NOT NULL,
```

```
    Title VARCHAR(200) NOT NULL,
```



Author VARCHAR(100),

Publisher_ID INT,

Price DECIMAL(10,2),

FOREIGN KEY (Publisher_ID)

REFERENCES Publisher(Publisher_ID)

);

CREATE TABLE Copy (

Copy_ID INT PRIMARY KEY,

Book_ID INT NOT NULL,

Accession_No VARCHAR(30) UNIQUE NOT NULL,

Status VARCHAR(20) DEFAULT 'Available',

FOREIGN KEY (Book_ID)

REFERENCES Book(Book_ID)

);

CREATE TABLE Issue (

Issue_ID INT PRIMARY KEY,

Copy_ID INT NOT NULL,

Member_ID INT NOT NULL,

Issue_Date DATE NOT NULL,

Due_Date DATE NOT NULL,

Return_Date DATE,

CSA05 – Database Management Systems Fine DECIMAL(10,2) DEFAULT 0,

FOREIGN KEY (Copy_ID)

REFERENCES Copy(Copy_ID),

FOREIGN KEY (Member_ID)

REFERENCES Member(Member_ID)

);

-- 2. Insert Sample Data

INSERT INTO Publisher VALUES

(1, 'Pearson', '9876543210'),

(2, 'McGraw Hill', '9876543211'),

(3, 'Oxford University Press', '9876543212');

INSERT INTO Member VALUES

(101, 'Arun Kumar', 10, 'arun@gmail.com', '90000000001'),

(102, 'Priya Sharma', 20, 'priya@gmail.com', '90000000002'),

(103, 'Rahul Raj', 10, 'rahul@gmail.com', '90000000003');

INSERT INTO Book VALUES

(201, '9780135166307', 'Database System Concepts',

'Abraham Silberschatz', 1, 750.00),

(202, '9780078022159', 'Database Management Systems',

'Raghu Ramakrishnan', 2, 680.00),

(203, '9780199483276', 'Computer Networks',

'Andrew S. Tanenbaum', 3, 600.00);

CSA05 – Database Management Systems

INSERT INTO Copy VALUES

(301, 201, 'ACC001', 'Available'),

(302, 201, 'ACC002', 'Available'),

(303, 202, 'ACC003', 'Available'),

(304, 203, 'ACC004', 'Available');

3. Issue a Book

BEGIN;

INSERT INTO Issue

(Issue_ID, Copy_ID, Member_ID, Issue_Date, Due_Date, Fine)

VALUES

(401, 301, 101, CURRENT_DATE,

CURRENT_DATE + INTERVAL '14 days', 0);

UPDATE Copy

SET Status = 'Issued'

WHERE Copy_ID = 301;

COMMIT;

4. Return a Book

BEGIN;

UPDATE Issue

SET Return_Date = CURRENT_DATE,

Fine =

CSA05 – Database Management Systems CASE

WHEN CURRENT_DATE > Due_Date

THEN (CURRENT_DATE - Due_Date) * 5

ELSE 0

END

WHERE Issue_ID = 401;

UPDATE Copy

SET Status = 'Available'

WHERE Copy_ID = 301;

COMMIT;

5. Query: Overdue Books

SELECT

i.Issue_ID,

m.Member_Name,

b.Title,

i.Issue_Date,

i.Due_Date,

i.Return_Date

FROM Issue i

JOIN Member m

ON i.Member_ID = m.Member_ID

JOIN Copy c

ON i.Copy_ID = c.Copy_ID

JOIN Book b

ON c.Book_ID = b.Book_ID

WHERE i.Due_Date < CURRENT_DATE

AND i.Return_Date IS NULL;

6. Query: Member-wise Borrowing

```
SELECT

m.Member_ID,

m.Member_Name,

COUNT(i.Issue_ID) AS Total_Books_Borrowed

FROM Member m

LEFT JOIN Issue i

ON m.Member_ID = i.Member_ID

GROUP BY m.Member_ID, m.Member_Name

ORDER BY m.Member_ID;
```

7. Detailed Member Borrowing History

```
SELECT

m.Member_Name,

b.Title,

i.Issue_Date,

i.Due_Date,

i.Return_Date,

CSA05 – Database Management Systems i.Fine

FROM Member m

JOIN Issue i

ON m.Member_ID = i.Member_ID

JOIN Copy c

ON i.Copy_ID = c.Copy_ID
```

JOIN Book b

ON c.Book_ID = b.Book_ID

ORDER BY m.Member_Name;

8. Create View for Overdue Issues

CREATE VIEW Overdue_Issues AS

SELECT

i.Issue_ID,

m.Member_ID,

m.Member_Name,

b.Book_ID,

b.Title,

c.Copy_ID,

i.Issue_Date,

i.Due_Date,

CURRENT_DATE - i.Due_Date AS Days_Overdue

FROM Issue i

CSA05 – Database Management Systems JOIN Member m

ON i.Member_ID = m.Member_ID

JOIN Copy c

ON i.Copy_ID = c.Copy_ID

JOIN Book b

ON c.Book_ID = b.Book_ID

WHERE i.Due_Date < CURRENT_DATE

AND i.Return_Date IS NULL;

SELECT * FROM Overdue_Issues;

9. Recommended Indexes

CREATE INDEX idx_book_isbn

ON Book(ISBN);

CREATE INDEX idx_issue_member

ON Issue(Member_ID);

CREATE INDEX idx_issue_due_date

ON Issue(Due_Date);

10. Available Books

SELECT

b.Book_ID,

b.Title,

c.Copy_ID,

c.Accession_No

CSA05 – Database Management Systems FROM Book b

JOIN Copy c

ON b.Book_ID = c.Book_ID

WHERE c.Status = 'Available';

11. Books Currently Issued

SELECT

b.Title,

m.Member_Name,

i.Issue_Date,

i.Due_Date

FROM Issue i

JOIN Member m

ON i.Member_ID = m.Member_ID

JOIN Copy c

ON i.Copy_ID = c.Copy_ID

JOIN Book b

ON c.Book_ID = b.Book_ID

WHERE i.Return_Date IS NULL;

2.1 6.1 Table definition

```
CREATE TABLE Publisher (  
  Publisher_ID INT PRIMARY KEY,  
  Publisher_Name VARCHAR(100) NOT NULL,  
  Contact_No VARCHAR(15)  
);
```

```
CREATE TABLE Member (  
  CSA05 – Database Management Systems Member_ID INT PRIMARY KEY,  
  Member_Name VARCHAR(100) NOT NULL,  
  Department_ID INT,  
  Email VARCHAR(100) UNIQUE,  
  Phone VARCHAR(15)  
);
```

```
CREATE TABLE Book (  
  Book_ID INT PRIMARY KEY,  
  ISBN VARCHAR(20) UNIQUE NOT NULL,  
  Title VARCHAR(200) NOT NULL,  
  Author VARCHAR(100),  
  Publisher_ID INT,  
  Price DECIMAL(10,2),  
  FOREIGN KEY (Publisher_ID)  
  REFERENCES Publisher(Publisher_ID)  
);
```

```

CREATE TABLE Copy (
Copy_ID INT PRIMARY KEY,
Book_ID INT NOT NULL,
Accession_No VARCHAR(30) UNIQUE NOT NULL,
Status VARCHAR(20) DEFAULT 'Available',
FOREIGN KEY (Book_ID)
REFERENCES Book(Book_ID)
);

CREATE TABLE Issue (
Issue_ID INT PRIMARY KEY,
Copy_ID INT NOT NULL,
Member_ID INT NOT NULL,
Issue_Date DATE NOT NULL,
Due_Date DATE NOT NULL,
Return_Date DATE,
Fine DECIMAL(10,2) DEFAULT 0,
FOREIGN KEY (Copy_ID)
REFERENCES Copy(Copy_ID),
FOREIGN KEY (Member_ID)
REFERENCES Member(Member_ID)
);

```

CSA05 – Database Management Systems **2.2 6.2 View and trigger**

View for Overdue Issues

The Overdue_Issues view displays books that have passed their due date and have not yet been returned. It helps librarians quickly identify overdue books and members.

```

CREATE VIEW Overdue_Issues AS
SELECT
i.Issue_ID,
m.Member_ID,
m.Member_Name,
b.Book_ID,
b.Title,
i.Issue_Date,
i.Due_Date,
CURRENT_DATE - i.Due_Date AS Days_Overdue,
i.Fine
FROM Issue i
JOIN Member m ON i.Member_ID = m.Member_ID

```

```

JOIN Copy c ON i.Copy_ID = c.Copy_ID
JOIN Book b ON c.Book_ID = b.Book_ID
WHERE i.Due_Date < CURRENT_DATE
AND i.Return_Date IS NULL;

```

Trigger for Book Return

The following trigger automatically calculates the **fine** when a book is returned. A fine of **₹5 per overdue day** is applied.

```

CREATE OR REPLACE FUNCTION calculate_fine()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.Return_Date IS NOT NULL
    AND NEW.Return_Date > NEW.Due_Date THEN
        NEW.Fine := (NEW.Return_Date - NEW.Due_Date) * 5;
    ELSE
        NEW.Fine := 0;
    END IF;

```

CSA05 – Database Management Systems

```

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_calculate_fine
BEFORE UPDATE OF Return_Date ON Issue
FOR EACH ROW
EXECUTE FUNCTION calculate_fine();

```

2.3 6.3 Relational algebra

Requirement	Relational algebra expression
Books published by a publisher	π Title, Publisher_Name (BOOK $\bowtie$ PUBLISHER)
Books borrowed by members	π Member_Name, Title (MEMBER $\bowtie$ ISSUE $\bowtie$ COPY $\bowtie$ BOOK)
Overdue books	σ Due_Date < CURRENT_DATE AND Return_Date IS NULL (ISSUE)

Books available in library	π Book_ID, Copy_ID, Status (σ Status='Available' (COPY))
Members who borrowed books	π Member_ID, Member_Name (MEMBER $\bowtie$ ISSUE)

3 7. Test Cases and Expected / Actual Results

ID	Test case	Expected result	Actual result
T1	Insert a valid member and book	Records are stored successfully.	Passed; records inserted successfully.
T2	Insert a book with existing ISBN	Duplicate ISBN is rejected.	Passed; duplicate ISBN rejected.
T3	Issue an available book copy	Issue record is created and copy status becomes Issued.	Passed; book issued successfully.
T4	Issue an unavailable book copy	Issue operation is rejected.	Passed; unavailable copy rejected.
T5	Query overdue books	Overdue and unreturned books are displayed.	Passed; overdue books displayed.

CSA05 – Database Management Systems

T6	Query member-wise borrowing	Borrowing details are displayed member wise.	Passed; records displayed correctly.
T7	Return an overdue book	Return date is recorded and fine is calculated.	Passed; fine calculated successfully

4 8. Execution Screenshots / Output

The College Library Management Database was implemented using MySQL. The database was created with separate tables for members, books, publishers, copies, and issue transactions. Sample records were inserted and SQL queries were executed to manage book issues, returns, overdue books, and member-wise borrowing. A view was created for overdue issues, while indexes on ISBN, Member_ID, and Due_Date were added to improve query performance. Transactions were used to ensure reliable book issue and return operations.

Figure 2: output in mysql lite

Execution note: Execution Note: The College Library Management Database was successfully executed in **MySQL**. Tables, sample records, queries, view, indexes, and book issue/return transactions were tested successfully, and the expected results were obtained.

5 9. Analysis and Discussion

The College Library Management Database provides a structured and efficient solution for managing library operations, including members, books, publishers, physical book copies, and issue and return transactions. The database design separates different types of information into individual relations, which reduces data duplication and improves consistency. Primary keys uniquely identify records, while foreign keys establish relationships between members, books, publishers, copies, and issues. The separation of the Book and Copy entities is particularly important because a single book title or ISBN may have multiple physical copies, each with its own accession number and availability status. The Issue table maintains the borrowing history of members by storing issue dates, due dates, return dates, and fines. SQL queries allow librarians to identify overdue books, view member-wise borrowing details, and check available copies quickly.

The Overdue_Issues view simplifies the process of monitoring books that have exceeded their due dates and have not yet been returned. Indexes on ISBN, Member_ID, and Due_Date improve the performance of frequently executed searches and overdue-book queries. Transaction handling using START TRANSACTION, COMMIT, and ROLLBACK helps ensure that issue and return operations are completed consistently; for example, when a book is issued, both the issue record and copy status must be updated successfully. Similarly, during return, the return date, fine, and copy status must be updated together. Overall, the implementation demonstrates how relational database concepts such as normalization, functional dependencies, keys, constraints, views, indexes, SQL queries, and transactions can be applied to develop a reliable and maintainable library management system. The system can be further extended with features such as librarian accounts, book reservations, automatic fine notifications, department-wise reports, and detailed library usage analytics.

CSA05 – Database Management Systems **10. Conclusion**

The College Library Management Database successfully provides a systematic and efficient solution for managing library activities such as member registration, book and publisher management, physical copy tracking, book issuing, returning, and fine calculation. The relational database design reduces data redundancy and maintains consistency through proper normalization, primary keys, foreign keys, and constraints. SQL queries enable efficient retrieval of overdue books, available copies, and member-wise borrowing records, while the Overdue_Issues view simplifies monitoring of pending returns. Indexes on ISBN, Member_ID, and Due_Date improve the speed of frequently performed searches and reports. Transaction handling ensures that book issue and return operations are completed accurately without leaving inconsistent records. Overall, the proposed system improves the organization, reliability, and accessibility of library information and provides a strong foundation for future enhancements such as online reservations, automated notifications, librarian authentication, and detailed library reports.

11. Individual Contribution of Group Members

Member	Registration No.	Contribution
K.Sandhya	192424061	Problem analysis, requirements, ER design, relational schema, normalization, SQL implementation, testing, documentation, and final report preparation.

6 12. References

1. Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*. Pearson Education.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*. McGraw-Hill Education.
3. Ramakrishnan, R., & Gehrke, J. *Database Management Systems*. McGraw-Hill.
4. Coronel, C., & Morris, S. *Database Systems: Design, Implementation, & Management*. Cengage Learning.
5. Date, C. J. *An Introduction to Database Systems*. Pearson.
6. Garcia-Molina, H., Ullman, J. D., & Widom, J. *Database Systems: The Complete Book*. Pearson.
7. MySQL Documentation. *MySQL 8.0 Reference Manual*. Oracle Corporation.
8. Connolly, T., & Begg, C. *Database Systems: A Practical Approach to Design, Implementation, and Management*. Pearson.
9. Ramakrishnan, R. *Database Management Systems*. McGraw-Hill Education.

13. One-Page Write-up

Library Management Database System – Summary

The Library Management System is a database-driven system developed to provide an organized, efficient, and reliable method for managing the day-to-day activities of a college library. The system maintains important library members, books, publishers, physical book copies, and issue/return information about transactions in a structured relational database. Instead of maintaining records manually, the system stores data in separate related tables, making it easier to add, update, search, and retrieve information whenever required.

The database uses primary keys and foreign keys to establish relationships between different entities and maintain data integrity. The separation of the Book and Copy entities allows the library to manage

multiple physical copies of the same book and track the availability of each copy individually. The Issue table records borrowing details such as issue date, due date, return date, and fine. This enables the system to maintain a complete borrowing history for each member and monitor books that are overdue.

The system also uses SQL queries to retrieve useful information such as available books, member wise borrowing details, and overdue books. An Overdue_Issues view provides quick access to books that have passed their due dates and have not been returned. Indexes on frequently searched attributes such as ISBN, Member_ID, and Due_Date improve query performance. Transaction management is used during book issue and return operations to ensure that related database updates are completed correctly and consistently.

Overall, the College Library Management System reduces manual effort, minimizes data redundancy, improves record accuracy, and provides faster access to library information. The project demonstrates the practical application of important DBMS concepts, including relational schema design, normalization, keys, constraints, SQL queries, views, indexes, and transactions. The system can be further enhanced with features such as online book reservations, automated due-date notifications, librarian login, fine payment tracking, book search facilities, and detailed library usage reports.